WEB-BASED TRUTH TABLE GENERATOR

Instant Evaluation & Visualization of Logical Expressions

Team Members :-

1. Krish Khajuria :- 2410111909
2. Arnav Vithlani :- 2410111185

INTRODUCTION & PURPOSE

PROJECT OVERVIEW

The Truth Table Generator is a web-based tool designed to instantly evaluate and visualize logical expressions. Built with a modern, dark-themed user interface, the application features dedicated input and output cards for seamless interaction. Key capabilities include real-time variable detection as users type their expressions, and color-coded truth tables that provide immediate visual clarity for logical results.

OBJECTIVES

User-Friendly Input Interface: Provide an intuitive input field with quick-insert buttons for logical operators (AND, OR, NOT), enabling users to build expressions effortlessly without memorizing syntax.

Real-Time Variable Detection: Automatically detect and display variables as the user types their logical expression, providing immediate feedback and confirming the system recognizes all input components.

Color-Coded Truth Table Output: Generate clear, readable truth tables with distinct visual styling—green for 'T' (True) and red for 'F' (False)—ensuring immediate comprehension of logical evaluation results.

TECHNOLOGIES USED

HTML5

Provides the foundational structure for the application using a clean, card-based layout. Semantic HTML elements organize the input card, variables display, and truth table output into distinct, well-structured sections.

CSS3

Delivers the modern dark gradient background with hover effects and distinct coloring for logical states. Green highlights for 'True' values and red for 'False' provide immediate visual feedback and clarity.

JS

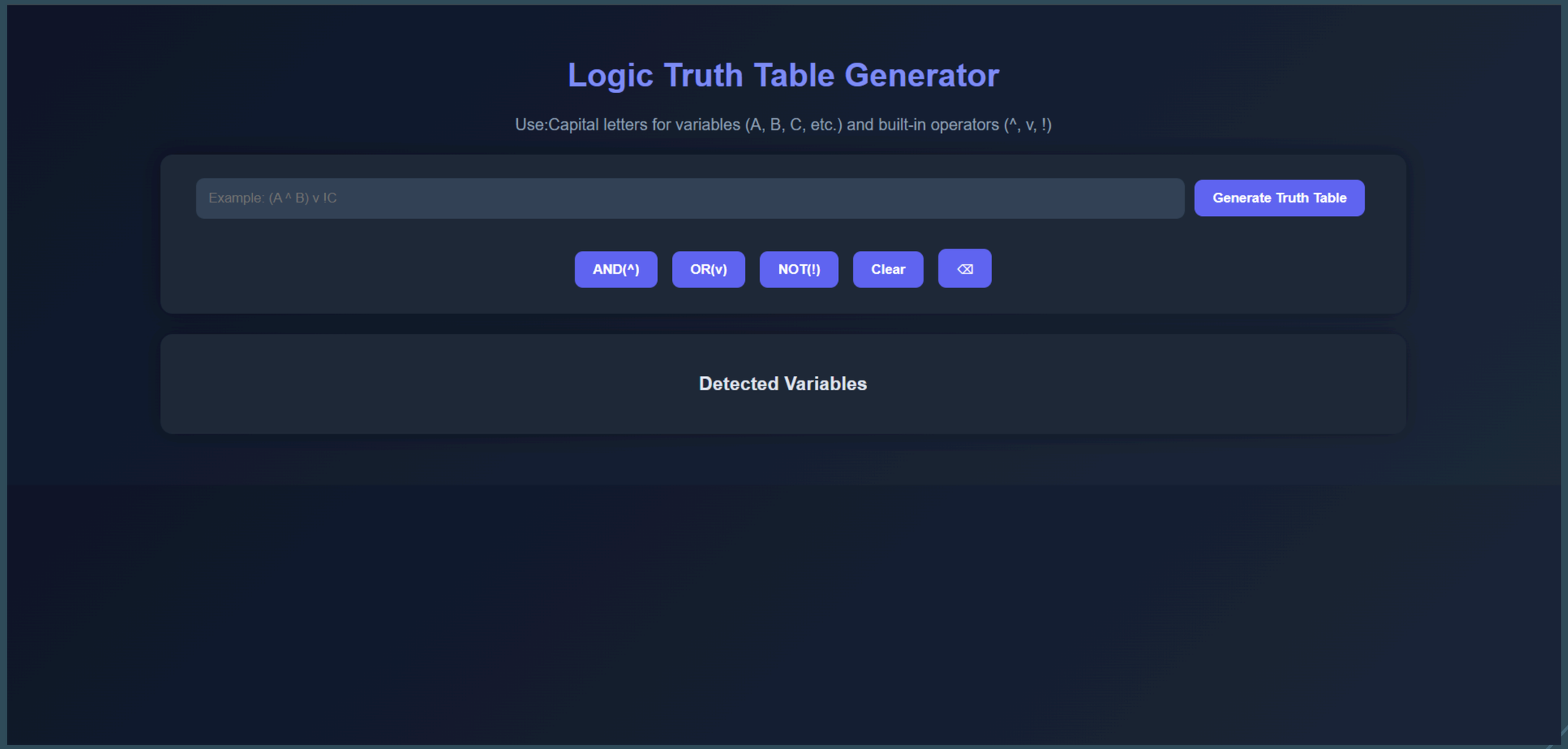
Drives the core logic and dynamic interactivity of the application. It utilizes Regular Expressions for real-time variable detection, performs bitwise operations to calculate truth value combinations, and handles DOM manipulation to seamlessly generate the output tables and manage error states without reloading the page.

PROJECT SCREENSHOT

APPLICATION INTERFACE

The Truth Table Generator features a sleek, dark-themed user interface designed for clarity and ease of use. The layout includes a dedicated input card where users enter logical expressions, with quick-insert buttons for common operators. Detected variables are displayed as pill-shaped badges below the input field for instant feedback. The truth table output showcases color-coded cells with green 'T' for True and red 'F' for False values, providing immediate visual comprehension of logical results.

PROJECT SCREENSHOT



CORE LOGIC OVERVIEW

Expression parsed to detect variables and operators using Regular Expressions. The parser scans input strings with regex pattern /[A-Za-z]+/g to identify unique alphabetical variables, building a dynamic variable list for table generation.

Operators mapped to JavaScript equivalents for evaluation. Logical symbols are converted: ^ → && (AND), v → || (OR), ! → ! (NOT) using .replaceAll() method, enabling native JavaScript logical operations on expressions.

Logical evaluation with binary state calculations and dynamic styling. Total combinations calculated as 2^n variables, bitwise operations generate binary states per row, and results receive CSS classes for color-coded T/F visualization.

CODE SNIPPET – EXPRESSION PARSER

Main.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Truth Table Generator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">

  <h1>Logic Truth Table Generator</h1>
  <p class="subtitle">
    Use:Capital letters for variables (A, B, C, etc.) and built-in operators (^, v, !)<br>
  </p>

  <!-- Input -->
  <div class="card">
    <input type="text" id="expression" placeholder="Example: (A ^ B) v !C" oninput="showVariables()">
    <button onclick="generateTable()">Generate Truth Table</button>

    <!-- Keyboard -->
    <div class="keyboard">
      <button onclick="insertValue('^')">AND(^)</button>
      <button onclick="insertValue('v')">OR(v)</button>
      <button onclick="insertValue('!')">NOT(!)</button>
      <button onclick="location.reload()">Clear</button>
      <button onclick="backspace()">⌫</button>
    </div>
  </div>

  <!-- Variables -->
  <div class="card">
    <h3>Detected Variables</h3>
    <div id="variables"></div>
  </div>

  <!-- Output -->
  <div class="card hidden" id="resultBox">
    <h3 id="title"></h3>
    <div id="output"></div>
  </div>

</div>
```

TRUTH TABLE GENERATION LOGIC

COMPUTATION :

Calculate total combinations as 2^(number of variables) to determine all possible input states. Use bitwise operations to generate binary states for each variable per row, enabling systematic enumeration of every logical combination.

STYLING :

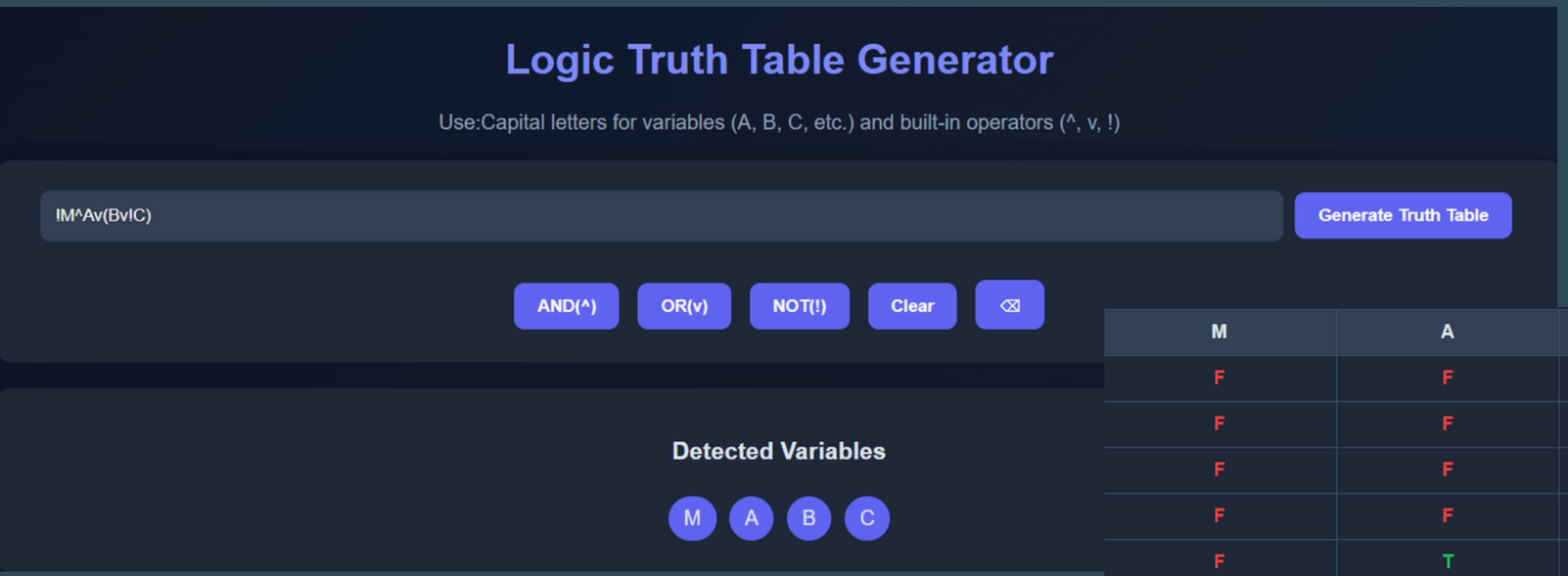
Convert binary 1/0 values to 'T'/'F' with CSS classes "true" (green) and "false" (red) for cells. Table rows highlight subtly on hover for improved usability and visual feedback during review.

OUTPUT EXAMPLE

DYNAMIC TABLE ADAPTATION

Headers dynamically adjust to detected variables (e.g., A, B, C) based on the logical expression entered. Color-coded results provide instant visual feedback: bold green cells display 'T' for True values, while bold red cells show 'F' for False values. Table rows feature a subtle background highlight on hover, enhancing usability and making it easy to track across columns. The responsive design ensures the truth table scales appropriately regardless of the number of variables detected.

OUTPUT EXAMPLE



M	A	B	C	Result
F	F	F	F	T
F	F	F	T	F
F	F	T	F	T
F	F	T	T	T
F	T	F	F	T
F	T	F	T	T
F	T	T	F	T
F	T	T	T	T
T	F	F	F	T
T	F	F	T	F
T	F	T	F	T
T	F	T	T	T
T	T	F	F	T
T	T	F	T	F
T	T	T	F	T
T	T	T	T	T

ERROR HANDLING

Variable Detection Alert: Before generating the truth table, the system checks if any variables were detected in the expression. If no variables are found, an alert notifies the user to enter a valid logical expression, preventing empty or meaningless table generation.

Try-Catch Protection: All evaluation logic is wrapped in try...catch blocks to gracefully handle invalid syntax or malformed expressions. This prevents JavaScript runtime errors from crashing the application and ensures the UI remains stable and responsive.

Graceful Error Returns: When evaluation fails due to syntax errors, the system returns an "Error" string instead of breaking the UI. This approach maintains visual consistency in the truth table output while clearly indicating which expressions could not be evaluated.

LEARNING OUTCOMES

- Advanced DOM Manipulation: Mastered dynamic UI updates to reveal results and modify content in real-time without page reloads.

- Regular Implementation: Developed pattern matching skills for variable detection using /[A-Za-z]+/g to extract unique alphabetical variables from expressions.

- Modern CSS Design: Applied gradients, box shadows, and dynamic styling techniques to create a polished, professional dark-themed interface.

- JavaScript Logic: Strengthened understanding of bitwise operations, logical evaluation, and safe code execution using Function constructor.

- Error Handling: Implemented robust try...catch blocks to gracefully manage invalid syntax and provide user-friendly feedback.

- User Experience: Learned to design intuitive interfaces with color-coded outputs and hover effects for enhanced readability.

FUTURE SCOPES

- Expanded Logic Gates: Adding buttons and parsing support for Implication and Biconditional operators.
- Regex Standardization: Unifying the regex logic (currently /[A-Za-z]+/g for calculation and /[A-Z]/g for display) to seamlessly handle both uppercase and lowercase variables across the entire application.
- Equation Exporting: Allowing users to download the generated HTML table as a CSV or image file.

THANK YOU!